Programación y pensamiento computacional

Nombre: _____ Fecha: _____

Qué vas a aprender

- Aplicar los cuatro pilares del pensamiento computacional a un problema
- Escribir algoritmos con secuencias, condiciones y bucles
- Crear y depurar programas sencillos por bloques o en texto
- Usar variables, funciones y listas para escribir programas ordenados
- Comparar algoritmos de búsqueda y ordenación

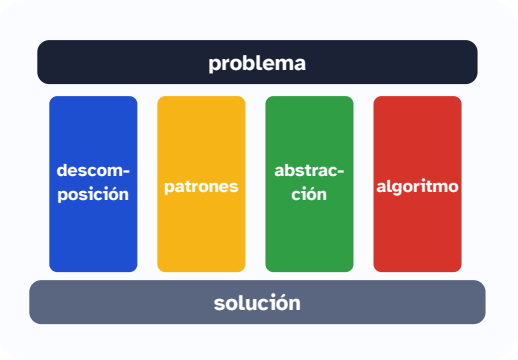
Para pensar

¿Cómo le explicarías a un robot que no sabe nada cómo preparar un bocadillo, si solo entiende órdenes muy simples?

Explicación

Los cuatro pilares

El **pensamiento computacional** es una forma de resolver problemas que se puede aplicar con o sin ordenador. Tiene cuatro pilares: **descomposición** (dividir un problema grande en partes pequeñas), **reconocimiento de patrones** (ver qué se repite o se parece a algo ya resuelto), **abstracción** (quedarse con lo importante e ignorar los detalles) y algoritmo (escribir los pasos ordenados para resolverlo).



Algoritmos y diagramas de flujo

Un **algoritmo** es una secuencia finita y ordenada de instrucciones que resuelve un problema. Se puede escribir en lenguaje natural, en **pseudocódigo** o con un diagrama de flujo, donde los óvalos marcan el inicio y el fin, los rectángulos las acciones, los rombos las decisiones (sí/no) y los paralelogramos la entrada y salida de datos. Un buen algoritmo es **preciso** (sin ambigüedades), **finito** (termina) y da siempre el mismo resultado con los mismos datos.



Las estructuras básicas

Cualquier programa se construye combinando tres estructuras. La **secuencia**: instrucciones una detrás de otra. La **selección** o **condicional**: si se cumple algo, haz esto; si no, lo otro. Y la **repetición** o **bucle**: repetir unas instrucciones un número de veces o mientras se cumpla una condición. Además se usan **variables**, cajas con nombre que guardan datos (puntos, vidas, nombre del jugador), y **operadores** para comparar y calcular.



Variables y tipos de datos

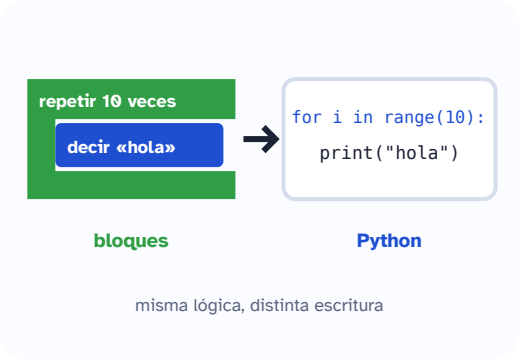
Una **variable** es un espacio de memoria con nombre donde el programa guarda un dato que puede cambiar: los puntos de un juego, la edad de un usuario, un nombre. Los datos tienen un **tipo**: **enteros** (7), **decimales** (3.5), **textos** o cadenas ("Hola"), **booleanos** (verdadero o falso) y **listas** que agrupan varios valores ([4, 8, 15]). Con ellos se hacen operaciones aritméticas (+, –, *, /), comparaciones (==, <, >) y operaciones lógicas (y, o, no). Elegir nombres claros para las variables, como `puntos` en vez de `x`, hace el programa mucho más fácil de entender.

Tipo	Ejemplo	Uso
Entero	15	Contar
Decimal	9.75	Medir
Texto	"Ana"	Nombres, mensajes
Booleano	True / False	Condiciones
Lista	[3, 7, 2]	Grupos de datos

undefined

De los bloques al código

Para empezar se usan lenguajes **por bloques**, como Scratch o MakeCode, donde las instrucciones encajan como piezas de puzle y no hay errores de escritura. Después se pasa a lenguajes **de texto**, como Python o JavaScript, más potentes. La lógica es la misma: el bloque «repetir 10 veces» se escribe en Python `for i in range(10):`. Los programas pueden responder a **eventos** (al pulsar una tecla, al tocar un objeto), lo que permite crear juegos, animaciones y aplicaciones.



Programar con Scratch

Scratch, creado en el MIT, permite programar encajando bloques de colores, sin errores de escritura. Cada **objeto** (sprite) tiene sus programas, que empiezan con un **evento** («al hacer clic en la bandera»). Los bloques se agrupan por categorías: movimiento, apariencia, sonido, control (bucles y condiciones), sensores, operadores y variables. Con ellos se crean juegos, animaciones e historias interactivas. Los objetos pueden comunicarse enviando **mensajes**. Es ideal para aprender la lógica, porque las estructuras son las mismas que en cualquier lenguaje: secuencia, condición y bucle.

undefined

Primeros pasos en Python

Python es uno de los lenguajes más usados del mundo, en ciencia, inteligencia artificial o páginas web, y su sintaxis es muy legible. `print("Hola")` muestra un texto; `nombre = input("¿Cómo te llamas? ")` pide un dato; `if nota >= 5:` abre una condición, y `for i in range(10):` repite diez veces. En Python la sangría (los espacios al principio de la línea) indica qué instrucciones están dentro de un bloque, así que es obligatoria. Los textos que llegan con `input` hay que convertirlos con `int()` si se quiere operar con ellos.

undefined

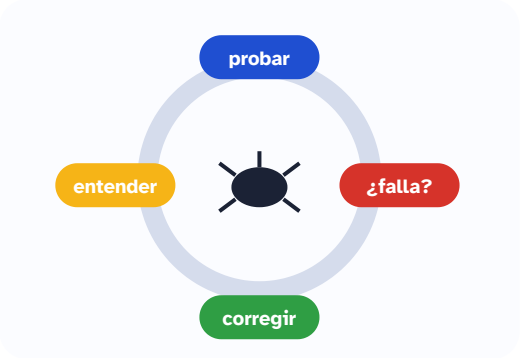
Funciones y modularidad

Una **función** es un trozo de programa con nombre que realiza una tarea concreta y se puede usar muchas veces. Recibe datos de entrada (**parámetros**) y puede devolver un resultado. Por ejemplo, una función `area_rectangulo(base, altura)` devuelve base · altura. Dividir un programa en funciones es aplicar la **descomposición**: cada parte es más fácil de escribir, probar y corregir, y se puede reutilizar en otros programas. En Scratch, las funciones se crean con «Mis bloques».

undefined

Probar y depurar

Casi ningún programa funciona a la primera. Los fallos se llaman **bugs**, y buscarlos y corregirlos es **depurar**. Hay errores de **sintaxis** (algo mal escrito, que impide ejecutar) y errores de **lógica** (el programa funciona pero hace otra cosa). Para depurar: lee el mensaje de error, prueba con datos sencillos cuyo resultado conoces, muestra el valor de las variables paso a paso y cambia una sola cosa cada vez. Explicar el código en voz alta (a un compañero o a un patito de goma) ayuda mucho.



Buscar y ordenar

Dos problemas clásicos muestran que hay algoritmos mejores que otros. Para **buscar** un dato en una lista desordenada solo cabe mirar uno por uno (**búsqueda lineal**). Pero si la lista está ordenada, la **búsqueda binaria** mira el elemento central y descarta la mitad en cada paso: en una lista de 1.000 elementos

encuentra el dato en unos 10 pasos en vez de hasta 1.000. Para **ordenar**, el método de la burbuja compara parejas vecinas y las intercambia si están desordenadas; es sencillo pero lento, y por eso se usan métodos más eficientes con muchos datos.

undefined

Pensamiento computacional en un problema real: organizar una excursión



Ejercicios

1. **Indica el tipo de dato: 42, "42", 3.14, True, [1, 2, 3].**

2. **¿Qué muestra este programa? a = 5; b = 3; a = a + b; print(a * 2).**

3. **Escribe un bucle en Python que muestre los números del 1 al 10.**

4. **Escribe un programa que pida la edad y diga «Puedes votar» si es 18 o más.**

5. **Escribe una función que reciba un número y devuelva su doble.**

6. **Describe un algoritmo para encontrar el número mayor de una lista.**

7. **En Scratch, ¿qué bloque usarías para que un objeto se mueva sin parar?**

8. **Encuentra el error: if nota > 5: print("Aprobado") (un alumno con un 5 aparece como suspenso).**

9. **Ordena con el método de la burbuja la lista [5, 2, 4, 1] e indica la primera pasada.**

10. **¿Por qué conviene dividir un programa en funciones?**

Practica

1. **Dividir un problema grande en partes pequeñas es...**

   A) abstracción
   B) descomposición
   C) depuración

2. **En un diagrama de flujo, una decisión se dibuja con...**

   A) un óvalo
   B) un rombo
   C) un rectángulo

3. **¿Qué estructura repite instrucciones?**

   A) El bucle
   B) La secuencia
   C) La variable

4. **¿Qué es una variable?**

   A) Un error del programa
   B) Un espacio con nombre que guarda un dato
   C) Un tipo de bucle

5. **Un programa se ejecuta pero suma en vez de restar. Es un error de...**

   A) sintaxis
   B) lógica
   C) hardware

6. **En Python, la sangría...**

   A) es opcional
   B) indica los bloques de código
   C) solo sirve para que quede bonito

7. **True y False son datos de tipo...**

   A) texto
   B) booleano
   C) entero

8. **== sirve para...**

   A) asignar

B) comparar

C) sumar

9. **La búsqueda binaria necesita que la lista esté…**

A) ordenada

B) vacía

C) en mayúsculas

10. **Una función que recibe datos los recibe como…**

A) parámetros

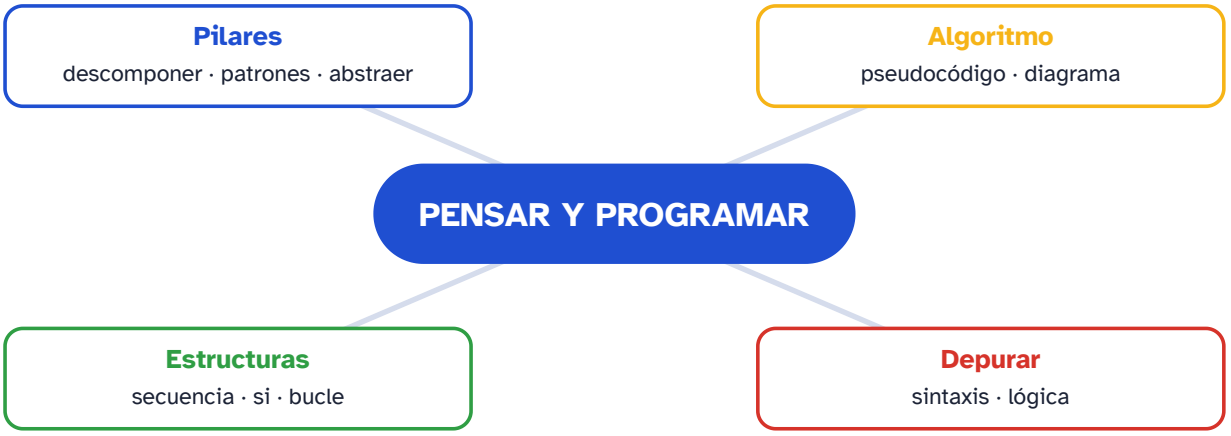
B) eventos

C) sprites

Explícalo con tus palabras

Incluye: Los cuatro pilares del pensamiento computacional con un ejemplo; Qué es un algoritmo y cómo se dibuja un diagrama de flujo; Secuencia, selección y bucle; Cómo pasar de un programa por bloques a Python; Errores de sintaxis y de lógica, y cómo depurarlos.

Resumen



Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Los cuatro pilares

Descomposición, patrones, abstracción y algoritmo.

¿Qué es un algoritmo?

Una secuencia finita y ordenada de instrucciones que resuelve un problema.

Las tres estructuras básicas

Secuencia, selección (condicional) y repetición (bucle).

¿Qué es depurar?

Buscar y corregir los errores (bugs) de un programa.

Error de sintaxis

Algo mal escrito que impide que el programa se ejecute.

Variable

Espacio de memoria con nombre que guarda un dato que puede cambiar.

Función

Trozo de programa con nombre que hace una tarea y se puede reutilizar.

Búsqueda binaria

Busca en una lista ordenada descartando la mitad en cada paso.

Soluciones

1. **B)** Cada parte es más fácil de resolver.
2. **B)** El rombo tiene dos salidas: sí y no.
3. **A)** Por ejemplo, «repetir 10 veces» o «mientras...».
4. **B)** Su valor puede cambiar durante el programa.
5. **B)** Está bien escrito, pero hace otra cosa.
6. **B)** Es obligatoria.
7. **B)** Verdadero o falso.
8. **B)** = asigna; == compara.
9. **A)** Descarta la mitad en cada paso.
10. **A)** Por ejemplo, base y altura.

Soluciones de los ejercicios

1. Entero, texto, decimal, booleano y lista.
2. a vale 8, así que muestra 16.
3. for i in range(1, 11): print(i).
4. edad = int(input("Edad: ")); if edad >= 18: print("Puedes votar").
5. def doble(n): return n * 2.
6. Guarda el primero como mayor; recorre la lista y, si un número es mayor que el guardado, sustitúyelo; al final, el guardado es el mayor.
7. Un bloque «por siempre» con «mover 10 pasos» y «rebotar si toca un borde» dentro.
8. La condición debe ser nota >= 5, para incluir el 5.
9. Primera pasada: [2, 5, 4, 1] → [2, 4, 5, 1] → [2, 4, 1, 5]. Al final: [1, 2, 4, 5].
10. Porque cada parte es más fácil de escribir, probar y corregir, y se puede reutilizar.